

Unidade I

1 CONCEITOS DE MODELAGEM DE BANCO DE DADOS RELACIONAL

Modelagem de banco de dados relacional estabelece a estrutura lógica e organizada para o armazenamento, o acesso e a manipulação dos dados. Ela visa representar de forma abstrata os elementos do mundo real que serão registrados no banco de dados, garantindo integridade, consistência e eficiência no tratamento das informações.

No modelo relacional, os dados são organizados em tabelas (ou relações) que possuem linhas (tuplas) e colunas (atributos). Cada tabela representa uma entidade, ou seja, um objeto ou conceito do mundo real que possui significado para o sistema (como clientes, produtos ou pedidos). Os atributos representam as características dessas entidades (por exemplo, nome, data de nascimento, preço), enquanto as tuplas representam instâncias específicas das entidades (como um cliente específico ou um produto cadastrado). Cada tabela deve possuir uma chave primária (primary key), que identifica de forma única cada registro.

Relacionamentos entre entidades descrevem como as tabelas se conectam entre si, permitindo a associação de dados dispersos. Para representar esses vínculos, utiliza-se a chave estrangeira (foreign key), que estabelece uma referência entre a chave primária de uma tabela e um campo em outra tabela.

Modelagem relacional normalmente passa por etapas de abstração e refinamento, começando pelo modelo conceitual de dados (como o Diagrama Entidade-Relacionamento – DER), avançando para o modelo lógico de dados (estruturado segundo os princípios do modelo relacional) e culminando no modelo físico de dados (implementado diretamente no sistema gerenciador de banco de dados – sistema gerenciador de banco de dados).

1.1 Modelagem Entidade-Relacionamento (ER)

Definição de entidades, relacionamentos e atributos

Compreender quais informações precisam ser armazenadas, qual o significado dessas informações e como elas se relacionam dentro do contexto do negócio. Essa

compreensão é alcançada por meio da modelagem conceitual, cujo objetivo principal é representar a realidade de forma abstrata, coerente e independente de tecnologias específicas, permitindo que os dados sejam organizados de acordo com regras claras e consistentes.

É essencial compreender as diferenças conceituais entre dado, informação, conhecimento e sabedoria. Esses elementos formam os degraus da chamada pirâmide DIKW (Data, Information, Knowledge, Wisdom), que representa a evolução da matéria bruta até a tomada de decisões sábias e fundamentadas.

Dado é definido como um elemento bruto, não interpretado, que representa um fato isolado ou um valor sem significado intrínseco.

Informação é resultante do processamento, da organização e da estruturação dos dados. Trata-se de dados contextualizados e compreensíveis, capazes de comunicar algo específico.

O conhecimento corresponde ao nível mais elevado da pirâmide e refere-se à aplicação da informação por meio da experiência, do raciocínio e da interpretação crítica. O conhecimento envolve julgamento, percepção e capacidade de agir com base nas informações disponíveis, sendo muitas vezes contextual, subjetivo e dependente do domínio do indivíduo.

A sabedoria representa o nível mais elevado da pirâmide. Ela corresponde à capacidade de utilizar o conhecimento com discernimento ético, visão crítica e sensibilidade contextual; visando ao bem comum ou à melhor decisão possível em situações complexas.



Figura 1 – Pirâmide DIKW

A modelagem conceitual assume papel ainda mais relevante, pois a qualidade dessa etapa influencia diretamente a estrutura das tabelas, a definição de chaves e a integridade dos dados.

As entidades representam os elementos do mundo real que possuem significado próprio dentro do domínio do problema e sobre os quais o sistema precisa armazenar informações. Esses objetos podem ser concretos, como pessoas, produtos ou veículos; ou abstratos, como contratos, pedidos ou matrículas.

No modelo relacional, cada entidade identificada na modelagem conceitual tende a ser posteriormente representada por uma tabela. Cada ocorrência da entidade corresponde a uma linha dessa tabela, enquanto cada atributo corresponde a uma coluna.

Dentro da modelagem relacional, é possível distinguir entidades que possuem existência independente daquelas que dependem de outras entidades para existir. As entidades com existência independente são aquelas que podem ser identificadas exclusivamente por seus próprios atributos, sem a necessidade de referência a outra entidade. Essas entidades, frequentemente chamadas de entidades fortes, representam objetos que existem por si mesmos no domínio do problema. Um cliente em um sistema comercial, por exemplo, existe independentemente de realizar compras, assim como um produto existe mesmo que ainda não tenha sido vendido.

Entidades cuja existência está diretamente vinculada a outra entidade. Essas entidades não têm significado completo de forma isolada e dependem de uma entidade principal para serem identificadas corretamente. No contexto do modelo relacional, essas entidades são frequentemente chamadas de entidades fracas. Um exemplo clássico ocorre em sistemas de vendas, nos quais o item de pedido depende da existência de um pedido. Um item de pedido não faz sentido sem estar associado a um pedido específico

A identificação de entidades fracas exige atenção especial, pois sua chave primária geralmente é composta pela chave da entidade forte à qual estão associadas, combinada com algum atributo adicional que permita distinguir as ocorrências.

A definição dos atributos deve ser realizada com critério, considerando quais informações são realmente necessárias para atender aos requisitos do sistema.

- Atributos simples: não podem ser decompostos, como “CPF”.
- Compostos: formados por outros atributos, como “Endereço”, que pode conter “Rua”, “Número” e

“Bairro”. • Atributos monovalorados: assumem um único valor, como “CPF”. • Atributos multivalorados: podem conter múltiplos valores, como “Telefones”. • Atributos derivados: cujo valor pode ser calculado a partir de outros atributos, como “Idade”, derivada da “Data de Nascimento”. • Atributo nulo: pode não ter um valor atribuído

Atributos derivados, que desempenham papel importante na modelagem conceitual, embora nem sempre sejam armazenados fisicamente no banco de dados relacional. Um atributo derivado é aquele cujo valor pode ser obtido a partir de outros atributos existentes. Um exemplo recorrente é a idade de uma pessoa, que pode ser calculada a partir da data de nascimento. Outro exemplo é o valor total de um pedido, que pode ser derivado a partir da soma dos valores dos itens que o compõem.

Atributo identificador, também conhecido como chave primária, a qual é o atributo, ou conjunto de atributos, responsável por identificar de forma única cada ocorrência da entidade, podem ser naturais ou artificiais. As chaves naturais utilizam atributos que já existem no mundo real e têm significado próprio, como o CPF de uma pessoa ou o número de matrícula de um aluno. Esse tipo de chave pode ser vantajoso por refletir diretamente a realidade do negócio, mas também pode apresentar desvantagens, como a possibilidade de mudanças no valor ou restrições legais relacionadas ao uso de dados sensíveis. As chaves artificiais são criadas especificamente para identificação, geralmente na forma de códigos numéricos sequenciais .

Em uma base de dados, cada ocorrência individual de uma entidade é chamada de instância. Assim, enquanto “Aluno” representa o tipo de entidade, um aluno específico, como “João da Silva, matrícula 20231234”, representa uma instância dessa entidade, os atributos assumem valores concretos para cada instância.

Os relacionamentos representam as associações existentes entre as entidades e expressam como os objetos do domínio do problema se conectam dentro do sistema. Enquanto as entidades representam os objetos e os atributos descrevem suas características, os relacionamentos indicam como esses objetos interagem entre si.

Um relacionamento pode ser entendido como uma regra do negócio que envolve duas ou mais entidades simultaneamente. Por exemplo, em um sistema de recursos humanos, existe um relacionamento entre funcionário e departamento, indicando que cada funcionário está alocado em um determinado departamento.

No banco de dados relacional, os relacionamentos são implementados por meio de chaves estrangeiras, as quais são atributos que referenciam a chave primária de outra tabela, estabelecendo um vínculo entre as entidades. No exemplo do relacionamento entre funcionário e departamento, a tabela funcionário pode conter um atributo que referencia a chave primária da tabela departamento.

Um relacionamento um-para-um ocorre quando cada ocorrência de uma entidade está associada a, no máximo, uma ocorrência da outra entidade.

O relacionamento um-para-um (1:1) ocorre quando cada instância de uma entidade está associada, no máximo, a uma única instância da outra entidade e vice-versa. Um exemplo típico é a relação entre “Campus” e “Diretor”: cada campus tem um único diretor e cada diretor está em um único campus.

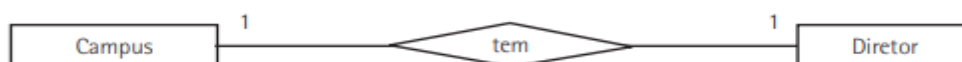


Figura 3 – Relação 1:1

O relacionamento um-para-muitos (1:N) é o mais comum nos sistemas de banco de dados relacionais. Nesse caso, uma instância de uma entidade A pode se relacionar com várias instâncias da entidade B, mas cada instância de B está associada a apenas uma instância de A. Um exemplo clássico é a relação entre “Departamento” e “Funcionário”: um departamento pode ter vários funcionários, mas cada funcionário pertence a apenas um departamento.

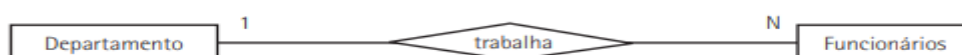


Figura 4 – Relação 1:N

O relacionamento muitos-para-muitos (N:N) ocorre quando múltiplas instâncias de uma entidade podem estar associadas a múltiplas instâncias de outra. Um exemplo recorrente é a relação entre “Aluno” e “Disciplina”: um aluno pode cursar várias disciplinas e uma disciplina pode ser frequentada por vários alunos. Diferentemente dos outros dois tipos, o relacionamento N:N não pode ser implementado diretamente em um banco de dados relacional, exigindo a criação de uma tabela associativa ou entidade de relacionamento que contenha as chaves primárias das duas entidades originais.



Figura 5 – Relação N:N

A entidade associativa representa explicitamente a relação entre duas entidades e permite armazenar informações adicionais sobre essa associação. No exemplo do relacionamento entre aluno e disciplina, a entidade associativa matrícula permite registrar dados como semestre, nota e situação do aluno.

Os relacionamentos também podem ser caracterizados quanto à obrigatoriedade da participação das entidades. Em alguns casos, a participação de uma entidade em um relacionamento é obrigatória, enquanto em outros é opcional. Por exemplo, em um sistema acadêmico, um aluno pode obrigatoriamente estar associado a um curso, enquanto um curso pode existir mesmo sem alunos matriculados temporariamente.

Embora os relacionamentos binários, que envolvem duas entidades, sejam os mais comuns no modelo relacional, existem situações em que um relacionamento envolve três ou mais entidades simultaneamente. Esses relacionamentos de grau superior representam associações mais complexas e exigem atenção especial durante a modelagem.

Em um relacionamento ternário, por exemplo, a associação só faz sentido quando as três entidades estão presentes simultaneamente. Um exemplo pode ser encontrado em sistemas de controle de projetos, nos quais um funcionário atua em um projeto desempenhando um determinado papel. Nesse caso, a relação entre funcionário, projeto e papel não pode ser adequadamente representada por relacionamentos binários independentes, pois o papel desempenhado depende da combinação específica de funcionário e projeto. No modelo relacional, esse tipo de relacionamento geralmente é representado por meio de uma entidade associativa que incorpora as chaves das entidades envolvidas.

Outro aspecto relevante na definição de relacionamentos diz respeito à dependência de existência entre entidades. Em alguns casos, a existência de uma entidade está condicionada à existência de outra, o que caracteriza uma relação de dependência forte. Esse tipo de dependência é comum em entidades fracas, que não têm significado completo sem a entidade à qual estão associadas.

A integridade referencial é o mecanismo responsável por assegurar que as referências entre tabelas sejam válidas. Ela garante que toda chave estrangeira corresponda a uma chave primária existente na tabela referenciada.

Domínio de valores de um atributo refere-se ao conjunto de valores válidos que ele pode assumir. Por exemplo, um atributo que representa o status de um pedido pode admitir apenas valores como “aberto”, “em processamento” e “finalizado”. A definição clara do domínio de valores contribui para a integridade dos dados e facilita a validação das informações inseridas no banco de dados. Essa definição começa na modelagem conceitual.

No modelo relacional cada entidade deve representar um conceito claramente definido, cada atributo deve ter um significado inequívoco e cada relacionamento deve expressar uma regra do negócio compreensível. Os relacionamentos, em particular, desempenham papel central na representação das regras do negócio. Eles não apenas conectam entidades, mas também expressam restrições e dependências que refletem a lógica do sistema. Por exemplo, em um sistema de controle acadêmico, o relacionamento entre aluno e curso pode indicar que um aluno deve estar vinculado a um curso para existir no sistema.

A definição de entidades fracas e fortes também influencia a forma como o sistema lida com a exclusão de dados. Quando uma entidade depende de outra para existir, a exclusão da entidade principal pode implicar a exclusão das entidades dependentes.

Os relacionamentos completam a estrutura conceitual do banco de dados relacional ao expressarem as associações entre as entidades. Eles traduzem regras do negócio e permitem que os dados sejam conectados de forma lógica e consistente.

Criação de DER

Esses diagramas são utilizados para representar graficamente a estrutura dos dados de um sistema, evidenciando as entidades envolvidas, seus atributos e os relacionamentos que as conectam. O principal objetivo do DER é fornecer uma visão clara e organizada da realidade que se deseja informatizar, permitindo que analistas, desenvolvedores e usuários compartilhem um entendimento comum sobre como as informações estão estruturadas e interligadas. O DER atua como uma ponte entre a análise de requisitos e a implementação do banco de dados relacional.

A construção de um DER inicia-se a partir da compreensão do domínio do problema. O analista deve estudar os processos do negócio, identificar quais informações são relevantes e entender como essas informações se relacionam. O resultado desse levantamento é um conjunto de conceitos que precisam ser representados no

diagrama, o próximo passo é identificar as entidades que farão parte do DER. As entidades representam os objetos ou conceitos do mundo real que têm significado próprio dentro do sistema e sobre os quais é necessário armazenar informações.

Dps o analista deve definir os atributos associados, os atributos representam as características das entidades e são responsáveis por armazenar os dados propriamente ditos. É importante na criação de DER é a identificação dos atributos que desempenham o papel de identificadores das entidades. Esses atributos, conhecidos como chaves primárias no contexto do modelo relacional.

A definição dos relacionamentos entre as entidades representam as associações existentes entre os objetos do domínio do problema e expressam regras do negócio que envolvem mais de uma entidade.

Durante a criação do DER, é necessário definir a cardinalidade dos relacionamentos, ou seja, quantas ocorrências de uma entidade podem estar associadas a ocorrências de outra entidade. A cardinalidade é um elemento essencial do diagrama, pois influencia diretamente a estrutura do banco de dados relacional que será implementado posteriormente.

Além da cardinalidade, a criação do DER envolve a definição da participação das entidades nos relacionamentos, indicando se essa participação é obrigatória ou opcional. Essa informação complementa a cardinalidade e fornece uma visão mais completa das restrições existentes no domínio do problema.

A cardinalidade indica quantas ocorrências de uma entidade podem estar associadas a ocorrências de outra entidade. Essa informação é crucial, pois influencia diretamente a estrutura lógica do banco de dados relacional que será derivado do diagrama.

Relacionamentos muitos-para-muitos por meio de entidades associativas. Embora no nível conceitual esse tipo de relacionamento possa ser representado diretamente, a criação de entidades associativas contribui para tornar o modelo mais claro e mais próximo da estrutura relacional que será implementada posteriormente. Uma entidade associativa representa a própria relação como um novo elemento do modelo, permitindo que atributos específicos dessa associação sejam representados de forma organizada. A utilização de entidades associativas no DER é importante quando o relacionamento possui atributos próprios ou a associação entre as entidades carrega significado relevante para o negócio.

1.2 Normalização de dados

Primeira, segunda e terceira formas normais (1FN, 2FN, 3FN)

Estabelece um conjunto de princípios que orientam a organização das tabelas de forma lógica, consistente e livre de redundâncias desnecessárias. O objetivo central da normalização é estruturar os dados de modo que cada informação seja armazenada no local mais apropriado, evitando problemas conhecidos como anomalias de inserção, atualização e exclusão, baseia-se na análise das dependências existentes entre os atributos de uma relação e na aplicação de regras conhecidas como formas normais. Cada forma normal estabelece critérios específicos que devem ser atendidos para que uma tabela seja considerada adequadamente estruturada em determinado nível. As formas normais são aplicadas de maneira progressiva, ou seja, para que uma tabela esteja na segunda forma normal, ela deve obrigatoriamente atender aos requisitos da primeira forma normal, e para que esteja na terceira forma normal, deve satisfazer as condições das duas anteriores.

1FN, representa o ponto de partida do processo de normalização. Uma tabela está na primeira forma normal quando todos os seus atributos possuem valores atômicos, ou seja, indivisíveis, e quando não existem grupos repetitivos ou atributos multivalorados armazenados em uma mesma coluna.

A segunda forma normal trata especificamente das dependências funcionais entre os atributos de uma tabela e tem como objetivo eliminar dependências parciais. Uma dependência parcial ocorre quando um atributo não chave depende apenas de parte da chave primária composta, e não da chave como um todo.

Para compreender a segunda forma normal, é necessário considerar o conceito de dependência funcional. Diz-se que um atributo depende funcionalmente de outro quando o valor de um determina unicamente o valor do outro. No contexto do modelo relacional, a chave primária determina todos os demais atributos da tabela. No entanto, em tabelas com chaves primárias compostas, podem existir atributos que dependem apenas de um dos componentes da chave, caracterizando uma dependência parcial.

Um exemplo clássico de violação da segunda forma normal pode ser observado em uma tabela que armazena informações sobre matrículas de alunos em disciplinas, utilizando como chave primária composta o identificador do aluno e o identificador da disciplina. Se essa tabela armazenar, além da nota do aluno na disciplina, informações como o nome do aluno ou o nome da disciplina, esses atributos dependerão apenas de parte da chave primária.

A terceira forma normal, conhecida como 3FN, aprofunda ainda mais o processo de normalização ao tratar das dependências transitivas. Uma dependência transitiva ocorre quando um atributo não chave depende de outro atributo não chave, que depende da chave

primária, na terceira forma normal, é necessário que todos os atributos não chave dependam diretamente e exclusivamente da chave primária, e não de outros atributos não chave. Quando existem dependências transitivas, a solução é decompor a tabela de forma a separar os atributos envolvidos em entidades distintas, preservando as dependências funcionais corretas.

Ao aplicar a 2FN, o analista passa a estruturar as tabelas de forma que cada uma represente um único conceito. A eliminação de dependências parciais contribui diretamente para a redução das chamadas anomalias de atualização. Quando um atributo depende apenas de parte da chave primária e está armazenado em uma tabela com chave composta, qualquer alteração nesse atributo precisa ser replicada em vários registros. Caso essa replicação não seja realizada de forma consistente, o banco de dados passa a conter informações divergentes sobre o mesmo elemento. É importante destacar que a segunda forma normal só se aplica a tabelas com chaves primárias compostas. Em tabelas cuja chave primária é formada por um único atributo, não há dependências parciais, pois todos os atributos dependem integralmente dessa chave. Nesses casos, a tabela que já está na primeira forma normal automaticamente satisfaz os requisitos da segunda forma normal.

Após a aplicação da segunda forma normal, o processo de normalização avança para a terceira forma normal, cujo foco está na eliminação de dependências transitivas. A terceira forma normal busca garantir que os atributos não chave dependam exclusivamente da chave primária. Uma dependência transitiva ocorre quando a chave primária determina um atributo não chave e este atributo determina outro atributo não chave.

As anomalias de inserção ocorrem quando a estrutura de uma tabela impede o cadastro de determinadas informações sem que outras estejam presentes. Em um modelo não normalizado, pode ser impossível registrar um novo produto sem que ele esteja associado a um pedido, ou cadastrar um novo departamento sem que exista ao menos um funcionário vinculado a ele.

As anomalias de atualização são talvez as mais conhecidas e ocorrem quando uma mesma informação é armazenada em múltiplos registros. Em uma tabela não normalizada, a alteração de um dado, como o endereço de um cliente ou o nome de um departamento, pode exigir a atualização de vários registros. Caso alguma dessas atualizações não seja realizada corretamente, o banco de dados passa a conter informações inconsistentes sobre o mesmo elemento.

As anomalias de exclusão, por sua vez, ocorrem quando a remoção de um registro resulta na perda de informações que deveriam ser preservadas. Em uma tabela que mistura dados de diferentes entidades, a exclusão do último registro relacionado a um determinado elemento pode apagar completamente informações importantes. Por exemplo, ao excluir o

último pedido de um cliente, pode-se perder também os dados do próprio cliente, caso essas informações estejam armazenadas na mesma tabela.

A aplicação progressiva de 1FN, 2FN e 3FN atua diretamente sobre essas anomalias, cada uma em um nível distinto. A primeira forma normal elimina problemas relacionados à atomicidade e aos grupos repetitivos, que dificultam a inserção e a atualização dos dados. A segunda forma normal trata das dependências parciais, reduzindo redundâncias que podem gerar inconsistências durante atualizações. A terceira forma normal elimina dependências transitivas, evitando a propagação de informações redundantes e prevenindo anomalias mais sutis.

Redução de redundâncias e prevenção de anomalia

A redundância ocorre quando a mesma informação é armazenada em mais de um local dentro do banco de dados. Embora, em alguns cenários específicos, a redundância controlada possa ser utilizada com objetivos de desempenho, sua presença excessiva ou não planejada tende a gerar inconsistências e dificuldades de manutenção. Quando um dado redundante precisa ser atualizado, todas as suas ocorrências devem ser modificadas de forma sincronizada. Caso isso não ocorra, o banco de dados passa a conter versões conflitantes da mesma informação, comprometendo sua integridade.

As anomalias de atualização são consequência direta da redundância. Quando uma informação é repetida em vários registros, qualquer alteração precisa ser propagada para todas as ocorrências. Se uma dessas atualizações não for realizada corretamente, o banco de dados passa a conter dados inconsistentes.

A prevenção de anomalias está intimamente ligada à redução da redundância por meio de uma modelagem adequada do banco de dados relacional. Esse processo envolve a identificação correta das entidades, a definição apropriada de atributos e a estruturação das relações de acordo com as dependências funcionais existentes.

Outro tipo de redundância está relacionado ao armazenamento de informações que podem ser derivadas a partir de outros dados já existentes no banco de dados. Quando valores derivados são armazenados de forma permanente, qualquer alteração nos dados de origem exige a atualização desses valores derivados. Caso essa atualização não ocorra de forma consistente, o banco de dados passa a conter informações contraditórias.

A relação entre redundância e dependências funcionais é um ponto central na compreensão das anomalias. As dependências funcionais descrevem como os atributos se relacionam entre si dentro de uma tabela, indicando quais atributos determinam outros. Quando essas dependências não são corretamente refletidas na estrutura das tabelas, a redundância tende a surgir. Por exemplo, se um atributo depende de outro atributo que não é chave e ambos estão armazenados na mesma tabela, cria-se uma situação propícia à redundância e às anomalias.

As anomalias clássicas de inserção, atualização e exclusão podem ser analisadas de forma mais detalhada à luz das dependências funcionais. A anomalia de inserção ocorre quando a estrutura da tabela impede a inserção de um dado sem que outro esteja presente, o que geralmente indica que informações de diferentes entidades estão misturadas. A redundância nesse caso se manifesta na forma de campos obrigatórios que não fazem sentido para determinados registros, evidenciando uma modelagem inadequada.

A anomalia de atualização está diretamente associada à repetição de dados. Quando um mesmo valor é armazenado em múltiplos registros, qualquer alteração exige atualizações múltiplas. A ausência de mecanismos automáticos que garantam a consistência dessas atualizações aumenta o risco de divergências.

A anomalia de exclusão ocorre quando a remoção de um registro resulta na perda de informações que deveriam ser preservadas. Esse problema é comum em tabelas que armazenam dados de diferentes entidades de forma conjunta. Ao excluir o último registro relacionado a um determinado elemento, todas as informações associadas a ele podem ser perdidas.

A prevenção de anomalias está relacionada à definição adequada das chaves primárias e estrangeiras. As chaves primárias garantem a unicidade dos registros, enquanto as chaves estrangeiras permitem relacionar tabelas sem necessidade de duplicação de dados.

As restrições de integridade referencial desempenham papel fundamental nesse processo, pois asseguram que todas as referências entre tabelas sejam válidas.

As restrições de integridade referencial desempenham papel fundamental nesse processo, pois asseguram que todas as referências entre tabelas sejam válidas. Ao impedir a inserção de valores que não correspondem a registros existentes em tabelas relacionadas.

A aplicação das formas normais atua como um guia prático para a redução de redundâncias. A primeira forma normal garante a atomicidade dos dados, eliminando grupos repetitivos e atributos multivalorados. A segunda forma normal elimina dependências parciais, evitando que atributos dependam apenas de parte de uma chave composta. A terceira forma normal trata das dependências transitivas, assegurando que cada atributo não chave dependa exclusivamente da chave primária.

O equilíbrio entre normalização e desempenho é uma questão central no projeto de bancos de dados relacionais. A redução de redundâncias deve ser realizada de forma consciente, considerando as características do sistema e os padrões de acesso aos dados. Em cenários nos quais determinadas consultas são executadas com alta frequência e exigem desempenho elevado, pode ser necessário avaliar a introdução controlada de redundância, desde que essa decisão seja tomada de forma planejada e documentada. Mesmo nesses casos, a normalização continua sendo o ponto de partida conceitual para o projeto.

A desnormalização, quando adotada, deve ser vista como uma exceção, e não como a regra. Ela consiste em reintroduzir redundância de forma controlada para atender a requisitos específicos de desempenho ou simplicidade de consulta. No entanto, essa prática aumenta a responsabilidade do desenvolvedor e do administrador do banco de dados, pois exige mecanismos adicionais para garantir a consistência dos dados redundantes. Sem esses mecanismos, as anomalias que a normalização busca evitar podem reaparecer.

A relação entre redução de redundâncias e desempenho merece atenção especial. Embora a eliminação da repetição de dados traga benefícios claros em termos de integridade e manutenção, ela pode implicar consultas mais complexas, que envolvem múltiplas junções. A avaliação desse trade-off deve ser realizada com base nos padrões de uso do sistema e nos requisitos de desempenho. Em muitos casos, o impacto das junções pode ser mitigado por meio de índices adequados e otimizações, preservando os benefícios da normalização sem comprometer significativamente a eficiência.

Quando a introdução controlada de redundância é considerada necessária, ela deve ser encarada como uma decisão consciente e bem documentada e não como uma solução improvisada. A desnormalização planejada pode ser utilizada para atender a requisitos específicos, desde que sejam implementados mecanismos que garantam

a consistência dos dados redundantes. Mesmo nesses cenários, a compreensão dos princípios de redução de redundâncias continua sendo fundamental, pois orienta a identificação dos pontos em que a redundância pode ser introduzida com menor risco.

2 DESNORMALIZAÇÃO E OTIMIZAÇÃO NO SQL SERVER

A desnormalização e a otimização de desempenho representam etapas estratégicas no uso de bancos de dados relacionais em ambientes corporativos, principalmente quando se utilizam sistemas robustos como o SQL Server. Embora a normalização seja fundamental para garantir integridade, consistência e redução de redundâncias, cenários reais de produção frequentemente impõem requisitos de desempenho que exigem flexibilizações controladas desse modelo. Nesse contexto, a desnormalização surge como uma abordagem complementar, aplicada de forma criteriosa, com o objetivo de reduzir a complexidade das consultas, minimizar operações de junção e melhorar o tempo de resposta das aplicações, sobretudo em ambientes com grande volume de dados e alto número de acessos simultâneos.

Recursos como índices, análise de planos de execução e técnicas de tuning de consultas permitem identificar gargalos, compreender o comportamento das consultas e ajustar a estrutura do banco de dados de acordo com os padrões reais de uso.

2.1 Vantagens e desvantagens da desnormalização no SQL Server

Impacto no desempenho e na escalabilidade

A desnormalização no contexto de bancos de dados relacionais representa uma abordagem consciente de flexibilização das regras de normalização com o objetivo de atender a requisitos específicos de desempenho, simplicidade de acesso ou escalabilidade. No SQL Server, essa prática assume particular relevância devido às características do sistema gerenciador, à forma como ele lida com consultas complexas, índices, paralelismo e grandes volumes de dados.

A desnormalização consiste na introdução controlada de redundância em um modelo de dados previamente normalizado. Enquanto a normalização busca eliminar redundâncias para garantir integridade e consistência, a desnormalização

parte do princípio de que, em determinados cenários, a repetição de dados pode trazer benefícios práticos, principalmente no que se refere à redução do custo de junções em consultas frequentes.

Ao desnormalizar determinadas estruturas, informações que antes estavam distribuídas em várias tabelas passam a ser armazenadas conjuntamente. Isso reduz a necessidade de junções e pode resultar em consultas mais simples e rápidas.

A escalabilidade é fator frequentemente associado à desnormalização. Em sistemas que precisam atender a um grande número de usuários simultâneos ou processar grandes volumes de dados, a redução do número de junções pode contribuir para melhor utilização dos recursos do SQL Server. Menos junções significam menos operações de leitura em diferentes páginas de dados e menor necessidade de sincronização entre threads, o que pode favorecer a escalabilidade horizontal e vertical do sistema.

A principal desvantagem da desnormalização é o aumento da redundância de dados, o que impacta diretamente a integridade e a consistência das informações. Quando um mesmo dado é armazenado em múltiplos locais, qualquer alteração precisa ser propagada para todas as ocorrências. Outro impacto importante da desnormalização está relacionado ao consumo de espaço em disco. A repetição de dados aumenta o volume total armazenado no banco de dados, o que pode impactar custos de armazenamento, estratégias de backup e tempos de restauração.

Em um ambiente no qual consultas complexas são executadas repetidamente, a desnormalização pode reduzir a necessidade de acessar múltiplas tabelas e de combinar grandes volumes de dados em tempo de execução. Ao concentrar dados frequentemente acessados em uma única tabela, o mecanismo de consulta pode gerar planos mais simples e eficientes.

Outro benefício importante da desnormalização está relacionado ao uso do cache de dados. O SQL Server utiliza intensivamente memória para armazenar páginas de dados frequentemente acessadas, reduzindo a necessidade de leituras em disco. Em modelos desnormalizados, uma única tabela pode conter a maior parte das informações necessárias para atender a uma consulta, aumentando a probabilidade de que esses dados estejam presentes no cache.

Em sistemas nos quais as consultas são altamente variadas ou em que os dados precisam ser atualizados com frequência, os ganhos podem ser menores ou até

inexistentes. A desnormalização é mais eficaz quando aplicada a conjuntos de dados relativamente estáveis e a consultas bem definidas, que justificam a introdução de redundância em troca de desempenho.

O aumento da complexidade das transações é outro fator relevante. Em modelos desnormalizados, as transações que envolvem atualização de dados redundantes tendem a ser mais longas e mais suscetíveis a conflitos. Isso pode resultar em maior contenção de locks, aumento de bloqueios e até mesmo em situações de deadlock.

A integridade dos dados é afetada pela desnormalização no que se refere à aplicação de restrições. Em modelos normalizados, as restrições de integridade referencial ajudam a garantir a consistência entre tabelas. Em modelos desnormalizados, parte dessa integridade precisa ser garantida por meio de lógica adicional na aplicação ou em procedimentos armazenados. Essa transferência de responsabilidade do banco de dados para a camada de aplicação aumenta a probabilidade de erros e torna o sistema mais dependente de implementações específicas.

A escalabilidade das operações de escrita é sensível à desnormalização. Em sistemas que precisam lidar com grande volume de transações simultâneas, a necessidade de atualizar dados redundantes pode limitar a capacidade de escalar horizontalmente ou verticalmente.

A comparação entre desnormalização e outras técnicas de otimização disponíveis no SQL Server é igualmente relevante. Muitas vezes, problemas de desempenho podem ser resolvidos por meio de indexação adequada, reescrita de consultas, uso de views indexadas ou ajustes de configuração, sem a necessidade de alterar a estrutura lógica do banco de dados. A desnormalização deve ser considerada quando essas alternativas não são suficientes para atender aos requisitos de desempenho ou quando os ganhos obtidos justificam claramente a introdução de redundância estrutural.

2.2 Ferramentas de otimização no SQL Server

Índices, planos de execução e tuning de consultas

Os índices representam uma das ferramentas mais importantes de otimização no SQL Server. Eles funcionam como estruturas auxiliares que permitem ao mecanismo de banco de dados localizar registros de forma mais eficiente, reduzindo a necessidade de varreduras

completas nas tabelas. Em essência, um índice organiza os dados de acordo com uma ou mais colunas, criando caminhos de acesso rápidos que podem ser explorados durante a execução das consultas.

Cada índice ocupa espaço em disco e exige manutenção durante operações de inserção, atualização e exclusão. No SQL Server, essas operações precisam manter os índices sincronizados com os dados da tabela, o que implica consumo adicional de CPU, memória e I/O.

SQL Server oferece diferentes tipos de índices, cada um adequado a cenários específicos. Índices clusterizados determinam a forma física de armazenamento dos dados na tabela, enquanto índices não clusterizados criam estruturas separadas que apontam para os registros. A escolha entre esses tipos influencia diretamente o desempenho das consultas e a organização interna do banco de dados. Um plano de execução representa a estratégia escolhida pelo otimizador do SQL Server para executar uma consulta. Ele descreve, de forma detalhada, quais operações serão realizadas, em que ordem e com quais métodos de acesso aos dados.

O otimizador de consultas do SQL Server utiliza estatísticas sobre a distribuição dos dados para escolher o plano de execução considerado mais eficiente. Essas estatísticas fornecem informações sobre o número de registros, a seletividade das colunas e outros aspectos relevantes. Quando as estatísticas estão desatualizadas ou imprecisas, o otimizador pode escolher planos subótimos, resultando em desempenho inferior.

A análise de planos de execução permite identificar operações custosas, como varreduras completas de tabelas, junções ineficientes ou uso inadequado de índices. Ao examinar esses planos, o profissional pode ajustar a estrutura do banco de dados ou reescrever consultas para orientar o otimizador a escolher estratégias mais eficientes.

O tuning de consultas complementa o uso de índices e a análise de planos de execução. Ele envolve a revisão e a reescrita das consultas SQL com o objetivo de torná-las mais eficientes. Pequenas alterações na forma como uma consulta é escrita podem ter impacto significativo no desempenho, especialmente em bancos de dados de grande porte.

No SQL Server, o tuning de consultas também está relacionado à compreensão do comportamento do otimizador. Embora o otimizador seja capaz de gerar planos eficientes na maioria dos casos, ele depende da forma como a consulta é expressa. Consultas ambíguas ou excessivamente complexas podem dificultar a escolha de

um plano ideal. Ao escrever consultas claras e alinhadas à estrutura dos dados, o desenvolvedor facilita o trabalho do otimizador e aumenta a probabilidade de obter bom desempenho.

Os índices clusterizados ocupam papel central na organização física das tabelas no SQL Server. Ao definir um índice clusterizado, os dados da tabela passam a ser armazenados fisicamente de acordo com a ordem das colunas que compõem esse índice. Isso significa que a escolha da chave do índice clusterizado influencia diretamente a forma como os dados são gravados e recuperados. Em consultas que utilizam filtros ou intervalos baseados nessa chave, o desempenho tende a ser significativamente melhor, pois o SQL Server pode localizar os registros de forma sequencial, reduzindo o número de leituras necessárias.

A definição inadequada de um índice clusterizado pode, no entanto, gerar efeitos negativos. Colunas que sofrem atualizações frequentes ou que não apresentam boa seletividade podem resultar em fragmentação e maior custo de manutenção. No SQL Server, a fragmentação de índices clusterizados pode aumentar o número de leituras necessárias para acessar os dados, afetando o desempenho das consultas. Por isso, a escolha da coluna ou conjunto de colunas que compõem o índice clusterizado deve considerar tanto o padrão de consultas quanto a frequência de alterações nos dados.

Os índices não clusterizados complementam o uso do índice clusterizado ao fornecer caminhos alternativos de acesso aos dados. Diferentemente do índice clusterizado, o índice não clusterizado é armazenado separadamente da tabela e contém referências aos registros correspondentes. Esses índices são úteis para consultas que filtram ou ordenam os dados por colunas que não fazem parte do índice clusterizado. Ao utilizar um índice não clusterizado adequado, o SQL Server pode evitar varreduras completas da tabela, reduzindo o custo das consultas. No entanto, o excesso de índices não clusterizados pode gerar sobrecarga significativa. Cada índice adicional exige manutenção durante operações de escrita, aumentando o custo de inserções, atualizações e exclusões. Em ambientes com alta taxa de escrita, esse custo pode tornar-se um fator limitante para o desempenho.

Índices compostos permitem indexar mais de uma coluna, o que pode ser útil em consultas que filtram ou ordenam os dados com base em múltiplos critérios. A ordem das colunas em um índice composto é fundamental, pois influencia quais consultas podem se beneficiar do índice. Um projeto inadequado pode resultar em índices que

raramente são utilizados, desperdiçando recursos. Os índices incluídos permitem adicionar colunas adicionais ao índice sem que elas façam parte da chave de indexação. Essa abordagem é útil para cobrir consultas específicas, permitindo que o SQL Server recupere todas as informações necessárias diretamente do índice, sem acessar a tabela base. Consultas cobertas tendem a apresentar melhor desempenho, pois reduzem o número de leituras de dados.

A análise dos planos de execução no SQL Server representa um dos momentos mais importantes do processo de otimização, pois é nesse artefato que se materializam as decisões tomadas pelo otimizador de consultas. Um plano de execução descreve, de forma detalhada, como o SQL Server pretende acessar os dados, quais operadores serão utilizados, a ordem das operações e as estimativas de custo associadas a cada etapa.

A relação entre planos de execução e tuning de consultas é, portanto, direta e complementar. O plano fornece evidências concretas sobre como o SQL Server está executando a consulta, enquanto o tuning envolve a aplicação de ajustes com base nessas evidências. Essa abordagem baseada em análise evita otimizações prematuras ou intuitivas, permitindo que as decisões sejam fundamentadas em dados reais sobre o comportamento do sistema. A prática contínua de análise de planos de execução contribui para o desenvolvimento de uma compreensão mais profunda do funcionamento interno do SQL Server.

Um dos princípios fundamentais do tuning de consultas é a redução do volume de dados processados em cada etapa da execução. Consultas que retornam ou manipulam mais dados do que o necessário tendem a consumir recursos excessivos, afetando o desempenho geral do sistema. A aplicação criteriosa de filtros, utilizando cláusulas que restringem os resultados o mais cedo possível no plano de execução, contribui para diminuir o número de linhas processadas nas etapas subsequentes. Essa prática é relevante em tabelas grandes, nas quais pequenas reduções no volume de dados podem resultar em ganhos significativos.

O uso adequado de expressões e funções nas consultas é outro aspecto importante do tuning. Funções aplicadas diretamente a colunas em cláusulas de filtro podem impedir o uso de índices, levando o SQL Server a realizar varreduras completas em vez de buscas eficientes. Ao reescrever essas expressões de forma que preservem a possibilidade de utilização dos índices, o desempenho pode ser significativamente melhorado.

Unidade II

3 MODELAGEM E GERENCIAMENTO NO SQL SERVER

Além da estruturação inicial dos dados, o gerenciamento eficaz do SQL Server envolve o domínio de consultas e operações avançadas capazes de extrair, manipular e processar informações de forma eficiente. Recursos como JOINS, subconsultas, funções agregadas e procedimentos armazenados permitem implementar regras de negócio, consolidar dados e otimizar o acesso às informações, reduzindo a sobrecarga na camada de aplicação.

3.1 Criação de esquemas e tabelas no SQL Server

Structured Query Language (SQL)

A SQL, ou linguagem de consulta estruturada, é um tipo de linguagem de consulta muito utilizada para executar operações em bancos de dados relacionais.

Tipos de linguagem SQL

- Data Definition Language (DDL): linguagem de definição de dados.
- Data Manipulation Language (DML): linguagem de manipulação de dados.
- Data Query Language (DQL): linguagem de consulta de dados.
- Data Control Language (DCL): linguagem de controle de dados.
- Data Transaction Language (DTL): linguagem de transação de dados.

Data Definition Language (DDL)

O subconjunto DDL permite criar, atualizar e remover tabelas e elementos associados. • Create: cria um objeto dentro do banco de dados (por exemplo: uma tabela). • Drop: remove um objeto do banco de dados. • Alter: altera um objeto do banco de dados (por exemplo: adicionando uma coluna a uma tabela existente).

Data Manipulation Language (DML)

O subconjunto DML é utilizado para realizar inclusões, consultas, alterações e exclusões de dados existentes em registros.

- Insert: insere um registro (tupla) em uma tabela do banco de dados.

INSERT INTO Funcionario

VALUES (1, 'Tadeu', 'Av.Paulista', 'Moema', 1500, 32);

GO

- Update: modifica os valores de uma ou várias linhas em uma tabela do banco de dados.

UPDATE Funcionario

SET Bairro = 'Ibira'

Where id_funcionario = 3;

- Delete: remove uma ou várias linhas em uma tabela do banco de dados

Podemos excluir apenas tuplas inteiras, e não apenas atributos. A SQL expressa a exclusão: DELETE FROM tabela WHERE [condição]

Data Query Language (DQL)

O subconjunto DQL permite ao usuário executar uma consulta (“query”) nas tabelas do banco de dados. O DQL é o mais utilizado, pois permite executar consultas das mais simples às mais elaboradas.

- Select: realiza consultas a dados específicos de uma ou várias tabelas.

Data Control Language (DCL)

O subconjunto DCL gerencia aspectos de autorização de dados e liberação de usuários para acessos de visualização ou manipulação de dados dentro do banco de dados. Os comandos utilizados são

- Grant: autoriza ao usuário executar determinada operação no banco.
- Revoke: remove a capacidade de um usuário de executar operações no banco.

Data Transaction Language (DTL)

O subconjunto DTL controla a execução de transações em um banco de dados. As transações são unidades de execução de programa que atualizam dados.

- Begin: marca o começo de uma transação de banco de dados que pode ser completada ou não (BEGIN TRANSACTION).
- Commit: finaliza uma transação que esteja sendo executada dentro de um sistema de gerenciamento de banco de dados.
- Rollback: faz com que as mudanças atuais nos dados existentes sejam abortadas, restaurando o banco até o estado do último COMMIT ou ROLLBACK.

Definição de tabelas, tipos de dados e relacionamentos

A criação de esquemas e tabelas no SQL Server representa uma etapa fundamental no processo de implementação de um banco de dados relacional, pois é nesse

momento que o modelo lógico é traduzido em estruturas físicas capazes de armazenar e organizar as informações de forma consistente.

O esquema no SQL Server funciona como um contêiner lógico que organiza objetos do banco de dados, como tabelas, visões e procedimentos. Ele permite agrupar objetos relacionados, facilitando a administração, o controle de permissões e a clareza estrutural do banco de dados.

Ao criar esquemas, o projetista estabelece uma camada adicional de abstração que separa logicamente os objetos sem a necessidade de criar bancos de dados distintos. Essa separação favorece a manutenção e a segurança, pois permite atribuir permissões específicas a grupos de objetos relacionados.

No contexto dos bancos de dados relacionais, a definição da estrutura que armazenará os dados é realizada por meio da linguagem de definição de dados – DDL, parte integrante do SQL. Entre os comandos da DDL, o mais fundamental é o CREATE TABLE, utilizado para criar uma tabela no banco de dados, especificando suas colunas, seus tipos de dados e suas restrições.

```
CREATE TABLE nome_da_tabela (  
    nome_coluna1 tipo_dado restrições_opcionais,  
    nome_coluna2 tipo_dado restrições_opcionais,  
    ... restrições_adicionais  
);
```

Cada linha dentro do parêntese define uma coluna da tabela, com seu respectivo tipo de dado (como INT, VARCHAR, DATE, entre outros) e possíveis restrições (como NOT NULL, UNIQUE, DEFAULT e PRIMARY KEY). Além disso, é possível definir restrições de chave estrangeira (FOREIGN KEY) para estabelecer vínculos com outras tabelas, promovendo a integridade referencial.

```
CREATE TABLE Clientes (  
    id_cliente INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE,  
    data_cadastro DATE DEFAULT CURRENT_DATE  
);
```

Além das chaves primárias, as chaves estrangeiras são fundamentais para a implementação dos relacionamentos entre tabelas. Elas permitem estabelecer

vínculos entre registros de diferentes tabelas, garantindo que as relações definidas no modelo lógico sejam respeitadas no nível físico.

Os tipos numéricos do SQL Server permitem armazenar valores inteiros e decimais com diferentes níveis de precisão e alcance. A escolha entre tipos inteiros menores ou maiores deve considerar o intervalo de valores esperado ao longo do tempo, evitando o uso de tipos excessivamente grandes quando não há necessidade. O uso de tipos adequados reduz o consumo de espaço em disco e melhora a eficiência das operações aritméticas, contribuindo para o desempenho geral do sistema. Tipos que armazenam valores aproximados podem introduzir imprecisões indesejadas, especialmente em cálculos acumulativos.

Os tipos de dados textuais são amplamente utilizados em tabelas que armazenam descrições, nomes, códigos e outros valores alfanuméricos. A definição correta do tamanho máximo dessas colunas é essencial para equilibrar flexibilidade e eficiência. Colunas excessivamente grandes podem desperdiçar espaço e impactar o desempenho, enquanto colunas muito restritivas podem impedir o armazenamento de informações legítimas.

Os tipos de dados temporais desempenham papel importante na representação de datas e horários, sendo amplamente utilizados em sistemas que registram eventos, transações e históricos. A escolha adequada desses tipos permite armazenar informações temporais com precisão apropriada, além de facilitar operações de comparação, ordenação e cálculo de intervalos.

A utilização de tipos binários é indicada para o armazenamento de dados não estruturados, como arquivos ou conteúdos codificados. No entanto, essa prática deve ser adotada com cautela, pois o armazenamento de grandes volumes de dados binários em tabelas pode impactar o desempenho e a manutenção do banco de dados. Em muitos casos, é mais adequado armazenar apenas referências a esses dados, mantendo-os em sistemas externos especializados.

A definição de valores nulos ou obrigatórios para cada coluna também está diretamente relacionada à escolha dos tipos de dados. Permitir valores nulos oferece flexibilidade, mas pode introduzir complexidade adicional nas consultas e nas regras de negócio. Por outro lado, exigir valores para determinadas colunas reforça a integridade dos dados, garantindo que informações essenciais estejam sempre presentes.

SQL Server permite estabelecer valores padrão para colunas, fornecendo automaticamente um valor quando nenhum é especificado durante a inserção.

A padronização dos tipos de dados entre tabelas relacionadas é essencial para a implementação correta dos relacionamentos. Colunas que participam de chaves primárias e estrangeiras devem utilizar tipos compatíveis, garantindo integridade referencial e evitando problemas de conversão.

A escolha criteriosa dos tipos de dados também influencia a evolução futura do banco de dados. Alterações posteriores em tipos de colunas podem ser complexas e impactar grandes volumes de dados, exigindo janelas de manutenção e cuidados adicionais.

A definição de chaves e restrições no SQL Server representa um dos pilares da criação de tabelas consistentes e alinhadas às regras do modelo relacional. Essas estruturas são responsáveis por garantir a integridade dos dados armazenados, assegurando que as informações respeitem critérios de unicidade, obrigatoriedade e coerência lógica. Ao implementar chaves e restrições de forma adequada, o projetista transfere para o próprio banco de dados parte significativa da responsabilidade pela validação dos dados, reduzindo a dependência de controles externos.

A chave primária é o elemento central na definição de uma tabela, pois identifica de forma única cada registro armazenado.

As restrições de unicidade complementam o papel da chave primária ao garantir que determinados valores não se repitam em uma ou mais colunas. Essas restrições são úteis para atributos que, embora não sejam identificadores principais, precisam manter valores únicos, como códigos ou identificadores alternativos.

As restrições de obrigatoriedade, implementadas por meio da proibição de valores nulos, reforçam a integridade dos dados ao assegurar que informações essenciais estejam sempre presentes. Colunas que representam informações fundamentais para a entidade devem ser definidas como obrigatórias, enquanto colunas opcionais podem permitir valores nulos, desde que isso esteja alinhado ao modelo conceitual.

Outro tipo importante de restrição é a que limita os valores permitidos em uma coluna com base em condições específicas. Essas restrições garantem que os dados armazenados atendam a critérios predefinidos, evitando a inserção de valores inválidos ou incoerentes.

A implementação de relacionamentos entre tabelas no SQL Server é realizada por meio de chaves estrangeiras, que estabelecem vínculos entre registros de diferentes tabelas. As chaves estrangeiras asseguram que os valores armazenados em uma tabela correspondem a registros válidos em outra, preservando a integridade referencial.

A implementação de relacionamentos de muitos para muitos requer atenção especial, pois envolve a criação de tabelas associativas que conectam as entidades envolvidas. Essas tabelas contêm chaves estrangeiras que referenciam as entidades principais e, frequentemente, possuem chaves primárias compostas.

A definição de restrições e relacionamentos também influencia o desempenho das operações no SQL Server. Embora essas estruturas imponham verificações adicionais durante inserções e atualizações, elas reduzem a necessidade de validações na camada de aplicação e previnem inconsistências que poderiam gerar problemas mais graves no futuro. O custo adicional das verificações é, na maioria dos casos, compensado pelos benefícios em termos de integridade e confiabilidade. Em sistemas com alta taxa de inserções ou atualizações, restrições excessivas ou mal definidas podem se tornar gargalos.

Os relacionamentos definidos por chaves estrangeiras também influenciam o comportamento das inserções. Ao exigir que valores referenciados existam previamente em outras tabelas, o SQL Server assegura a integridade referencial, mas impõe uma ordem lógica às operações. Essa dependência exige que registros relacionados sejam inseridos na sequência correta, o que deve ser considerado no desenvolvimento das aplicações e na carga inicial de dados. Quando essa ordem não é respeitada, as inserções falham, evidenciando a importância de um entendimento claro das relações entre as tabelas.

O impacto das definições estruturais sobre as operações de inserção, atualização e exclusão evidencia a importância de alinhar o modelo físico às regras de negócio. Restrições bem planejadas garantem que os dados sejam validados desde sua entrada no sistema, enquanto decisões conscientes sobre ações associadas a relacionamentos evitam comportamentos inesperados.

3.2 Consultas e operações avançadas em SQL Server

Estrutura básica de consultas SQL

• SELECT lista_atributos. • FROM tabela. • WHERE condição.

Comando SELECT

O retorno de uma consulta SQL é uma relação que lista todos os atributos selecionados no resultado de uma consulta. O comando SELECT facilita a projeção das informações, pois possibilita a eliminação de dados duplicados.

Para forçar a eliminação de nomes duplicados na tabela funcionário utilizamos o comando:

```
SELECT distinct NOME FROM Funcionario
```

```
GO
```

Cláusula WHERE

Para selecionar valores específicos de algum atributo, utilizamos a cláusula WHERE com o SELECT. Usamos os conectores and, or e not juntamente com os operadores de comparação (=, <>, >, >=, <, <=)

```
SELECT Nome FROM Funcionario WHERE Nome = 'Tadeu'
```

Utilizando operador de comparação between para facilitar as cláusulas WHERE que especifiquem um intervalo de valor (com mínimo e máximo):

```
SELECT Nome FROM Funcionario WHERE Nome between 'Joao' and 'Tadeu'
```

```
SELECT Nome FROM Funcionario WHERE Nome not between 'Joao' and 'Tadeu'
```

Usando o in:

```
SELECT Nome FROM Funcionario WHERE Nome in ('Joao', 'Tadeu')
```

```
SELECT Nome FROM Funcionario WHERE Nome not in ('Joao', 'Tadeu')
```

Usando o like:

```
SELECT Nome FROM Funcionario WHERE Nome like 'J%'
```

Usando o OR e AND:

```
SELECT Nome FROM Funcionario WHERE Nome = 'Tadeu' or Nome = 'Vania'
```

```
SELECT Nome FROM Funcionario WHERE Nome like 'J%' and Salario > 1000
```

Cláusula FROM

A cláusula FROM define um produto cartesiano das relações na cláusula.

Operações de conjunto

Quando vemos as funções dos conjuntos, lembramos de conjuntos da disciplina de matemática, em que poderíamos formar conjuntos de números combinando, separando e observando o que ambos têm em comum. No SQL há operações que levam esse conceito em consideração e o utilizam para agrupar dados que formam uma espécie de “conjuntos de dados”, ou seja, funções que agregam conjuntos de dados usando os mesmos princípios operacionais estudados na disciplina de matemática. Essas operações são: UNION, INTERSECT e EXCEPT.

Os operadores de conjunto agrupam os resultados da consulta de duas seleções em um único resultado. As consultas contêm operadores bem estabelecidos chamados consultas compostas. Com esses operadores, é possível combinar várias consultas e todos os operadores estabelecidos têm a mesma precedência.

Se as consultas retornarem valores do tipo de texto de comprimento igual, então os valores devolvidos têm tipo de texto equivalente. Se as consultas retornarem valores do tipo de texto e com diferentes comprimentos, o valor de retorno é texto variado e é considerado o comprimento do maior valor.

UNION

Diferentemente da operação SELECT, a operação UNION elimina os valores duplicados. Ela combina os resultados de duas linhas, eliminando as linhas duplicadas, ou seja, todas as linhas que aparecerão no resultado serão distintas.

Encontrar o nome de todos os Funcionários e o nome de todos os Clientes eliminando os nomes duplicados:

```
(select Nome from Funcionario)
```

```
union
```

```
(select Nome from Cliente)
```

Para encontrar o nome de todos os Funcionários e o nome de todos os Clientes mantendo os nomes duplicados utilizamos o UNION ALL:

```
(select Nome from Funcionario)
```

```
union all
```

```
(select Nome from Cliente)
```

INTERSECT

A operação INTERSECT encontra os registros que pertencem aos dois SELECTs, isto é, combina os resultados e retorna somente as linhas resultantes por todas as consultas. No entanto ele só retorna as linhas de conjuntos de dados caso um registro exista em uma consulta e nas demais também.

Para encontrar o nome de todos os Funcionários que também têm nome na tabela Clientes (sem duplicados):

```
(select Nome from Funcionario)
```

```
intersect
```

```
(select Nome from Cliente)
```

Para encontrar o nome de todos os Funcionários que também têm nome na tabela Clientes sem eliminar os nomes duplicados INTERSECT ALL:

```
(select Nome from Funcionario)
```

```
intersect all
```

```
(select Nome from Cliente)
```

EXCEPT

A operação EXCEPT encontra os registros que pertencem ao primeiro SELECT e não aparece no segundo. Para encontrar o nome de todos os Funcionários que não têm o nome na tabela Clientes (sem duplicados)

```
(select Nome from Funcionario)
```

```
except
```

```
(select Nome from Cliente)
```

Para encontrar o nome de todos os Funcionários que não têm o nome na tabela Clientes sem eliminar os nomes duplicados EXCEPT ALL:

```
(select Nome from Funcionario)
```

```
except all
```

```
(select Nome from Cliente)
```

Funções agregadas

As funções de agregação são funções da SQL que executam uma operação aritmética nos valores de uma coluna específica ou em todos os registros de uma tabela do banco de dados (COUNT, MAX / MIN, SUM e AVG)

- COUNT (contador de ocorrências [de um atributo]).

```
-- Apresenta a quantidade de funcionários
SELECT count(id_Funcionario) as QtTotal
FROM Funcionario
```

- MAX / MIN (valores máximos / mínimos de um atributo).

```
-- Apresenta o maior Salário
SELECT max(Salario) as Maior_Salario
FROM Funcionario
```

```
-- Apresenta o menor Salário
SELECT min(Salario) as Menor_Salario
FROM Funcionario
```

- SUM (somador de valores de um atributo).

```
-- Apresenta a soma dos Salários
SELECT sum(Salario) as Total
FROM Funcionario
```

- AVG (média de valores de um atributo).

```
-- Apresenta a média de Salário dos Funcionários
SELECT avg(Salario) as Media
FROM Funcionario
```

Valores nulos

A SQL permite a utilização de valores nulos para registrar a ausência de dado sobre o valor do atributo, utilizamos a palavra null para localizar esses registros.

```
SELECT Nome from Funcionario
```

```
WHERE Setor is null
```

Para localizar os funcionários que têm Setor, utilizamos o comando:

```
SELECT Nome from Funcionario
```

```
WHERE Setor is not null
```

Tipos de JOIN em SQL

Os comandos JOIN são utilizados para combinar registros de duas ou mais tabelas com base em uma condição lógica, geralmente relacionada por chaves primárias e estrangeiras. Os principais tipos são:

- INNER JOIN Retorna apenas os registros que têm correspondência em ambas as tabelas. Exemplo:

```
SELECT Cliente.nome, Pedido.data_pedido FROM Cliente INNER JOIN Pedido ON
Cliente.id_cliente = Pedido.id_cliente;
```

- LEFT JOIN (LEFT OUTER JOIN)

Retorna todos os registros da tabela da esquerda e os dados da tabela da direita apenas se houver correspondência. Onde não houver, retorna NULL.

```
SELECT Cliente.nome, Pedido.data_pedido FROM Cliente LEFT JOIN Pedido ON
Cliente.id_cliente = Pedido.id_cliente;
```

- **RIGHT JOIN (RIGHT OUTER JOIN)**

Faz o oposto do LEFT JOIN: retorna todos os registros da tabela da direita e os da esquerda apenas se houver correspondência. Exemplo:

```
SELECT Pedido.id_pedido,
Cliente.nome FROM Pedido RIGHT JOIN Cliente ON Pedido.id_cliente =
Cliente.id_cliente;
```

- **FULL JOIN (ou FULL OUTER JOIN)** Retorna todos os registros de ambas as tabelas, com correspondência onde possível e NULL onde não houver. Exemplo:

```
SELECT Cliente.nome, Pedido.data_pedido FROM Cliente FULL JOIN Pedido ON
Cliente.id_cliente = Pedido.id_cliente;
```

 Combina todos os clientes e os pedidos, mesmo que não estejam relacionados entre si.

- **CROSS JOIN** Retorna o produto cartesiano entre as duas tabelas: cada linha da primeira tabela é combinada com todas as linhas da segunda. Exemplo:

```
SELECT
Cliente.nome, Produto.nome FROM Cliente CROSS JOIN Produto
```

 Retorna todas as combinações possíveis entre clientes e produtos, útil em simulações ou análises combinatórias.

JOINS, subconsultas, funções agregadas e procedimentos armazenados

As consultas e as operações avançadas em SQL Server representam um estágio de maturidade no uso do banco de dados relacional, no qual o profissional deixa de apenas armazenar e recuperar dados básicos para explorar todo o potencial da linguagem SQL como ferramenta de análise, integração e automação.

Os procedimentos armazenados representam outro recurso avançado fundamental no SQL Server, permitindo encapsular lógica de consulta e manipulação de dados no próprio banco de dados. Ao utilizar procedimentos armazenados, o desenvolvedor transfere parte da lógica de processamento para o servidor, promovendo reutilização, segurança e desempenho. Esses procedimentos podem conter consultas complexas, operações condicionais e chamadas a funções, constituindo componentes essenciais de aplicações robustas.

Além do comportamento lógico, os JOINS exercem impacto significativo sobre o desempenho das consultas. Operações que combinam grandes volumes de dados podem consumir recursos consideráveis de CPU, memória e I/O, especialmente

quando não há índices adequados para suportar as colunas envolvidas na junção. No SQL Server, a eficiência dos JOINS depende fortemente da existência de índices compatíveis e da atualização das estatísticas utilizadas pelo otimizador de consultas. A utilização de filtros em conjunto com JOINS é uma prática essencial para controlar o volume de dados processados. Aplicar critérios de seleção adequados reduz o número de registros envolvidos nas junções, melhorando o desempenho e a relevância dos resultados. A posição lógica desses filtros na consulta influencia a forma como o SQL Server os aplica durante a execução, impactando o custo total da operação.

As subconsultas no SQL Server ampliam significativamente a expressividade da linguagem SQL ao permitir que uma consulta seja utilizada como parte integrante de outra, criando estruturas hierárquicas de consulta capazes de resolver problemas complexos de forma elegante. Esse recurso possibilita que resultados intermediários sejam calculados dinamicamente e utilizados como critérios de seleção, comparação ou fonte de dados, oferecendo uma alternativa conceitual aos JOINS em diversos cenários. A utilização de subconsultas é útil quando a lógica da consulta exige dependência direta entre conjuntos de dados. Em situações nas quais o resultado de uma operação precisa ser conhecido antes que outra seja avaliada, a subconsulta fornece um mecanismo natural para expressar essa dependência.

A integração dos procedimentos armazenados com JOINS, subconsultas e funções agregadas evidencia o caráter unificador desse recurso. Um único procedimento pode reunir dados de várias tabelas por meio de JOINS, aplicar filtros dinâmicos utilizando subconsultas e calcular indicadores por meio de funções agregadas, entregando resultados prontos para consumo pelas aplicações. Essa integração permite construir operações completas e eficientes, encapsulando toda a complexidade no nível do banco de dados.

4.1 Ferramentas de modelagem utilizadas no SQL Server

Utilização do SQL Server Management Studio (SSMS) para criação e gerenciamento de DER

A utilização do SSMS como ferramenta de apoio à criação e ao gerenciamento de DER representa uma etapa importante na consolidação da modelagem de dados em ambientes que utilizam o SQL Server. O SSMS não se limita a ser um ambiente para

execução de comandos SQL, mas oferece recursos gráficos que permitem visualizar, documentar e validar a estrutura do banco de dados relacional de forma integrada.

Os DER desempenham papel fundamental na representação visual do modelo de dados, pois permitem observar entidades, atributos e relacionamentos de forma clara e organizada. Ao utilizar o SSMS para criar esses diagramas, o projetista passa a trabalhar diretamente sobre o banco de dados existente, garantindo que a representação gráfica esteja alinhada com tabelas, chaves e restrições.

A criação de diagramas no SSMS exige compreensão prévia dos conceitos de entidade, atributo e relacionamento, pois a ferramenta apenas representa graficamente o que já foi definido no banco de dados. Dessa forma, o uso eficaz do recurso depende da qualidade da modelagem inicial e da correta definição das chaves e das restrições. Diagramas bem estruturados refletem modelos bem projetados.

DER no SSMS não substitui ferramentas especializadas de modelagem conceitual, mas complementa essas ferramentas no contexto da implementação e da manutenção. Enquanto o modelo conceitual é geralmente criado em etapas iniciais do projeto, o diagrama no SSMS atua como uma representação do estado atual do banco de dados, sendo útil em fases de manutenção, evolução e auditoria do sistema.

SSMS oferece, portanto, uma ponte entre o modelo relacional teórico e sua implementação prática.

A seleção das tabelas que compõem o diagrama é um passo decisivo para a clareza da representação. Em bancos de dados extensos, incluir todas as tabelas em um único diagrama pode resultar em uma visualização excessivamente complexa e de difícil interpretação. Por isso, é comum criar diagramas que representem subconjuntos do modelo, agrupando tabelas relacionadas a um mesmo domínio funcional ou área de negócio. Essa prática melhora a legibilidade e torna o diagrama mais útil como ferramenta de análise e documentação.

Ao adicionar tabelas ao diagrama, o SSMS exibe automaticamente suas colunas e identifica visualmente as chaves primárias. Os relacionamentos entre as tabelas são representados graficamente por linhas que conectam as entidades, indicando a existência de chaves estrangeiras.

O SSMS representa apenas a estrutura física do banco de dados e não incorpora conceitos puramente conceituais, como generalizações ou heranças, comuns em modelos mais abstratos. Por isso, o diagrama deve ser interpretado como uma visão do modelo relacional implementado, e não como substituto completo de ferramentas de modelagem conceitual.

O SSMS não é uma ferramenta de modelagem conceitual completa e não oferece suporte nativo para representar abstrações mais avançadas, como herança ou especializações. Seu foco está na representação do modelo físico existente, o que limita sua aplicação em fases iniciais de concepção do sistema. Por isso, o SSMS deve ser visto como complemento, e não substituto, de ferramentas de modelagem conceitual.

Introdução ao uso de ferramentas como o Azure Data Studio para visualização e manipulação de dados

A introdução ao uso de ferramentas modernas para visualização e manipulação de dados representa evolução natural no modo como profissionais interagem com bancos de dados e sistemas de informação. Em ambientes cada vez mais orientados a dados, não basta apenas armazenar informações de forma estruturada, é fundamental dispor de recursos que facilitem a exploração, a análise e a compreensão desses dados de maneira eficiente e intuitiva. Ferramentas como o Azure Data Studio surgem nesse contexto como soluções que ampliam as possibilidades de interação com os dados, integrando consulta, visualização e gestão em um único ambiente.

A visualização de dados desempenha papel central nesse contexto, pois transforma conjuntos extensos de registros em representações gráficas capazes de revelar padrões, tendências e anomalias. Gráficos, tabelas dinâmicas e painéis interativos permitem compreender rapidamente informações que seriam difíceis de interpretar apenas por meio de consultas textuais.

A integração entre consultas SQL e visualizações favorece uma abordagem iterativa de análise. O usuário pode executar uma consulta, observar os resultados em forma gráfica, ajustar critérios ou agregações e imediatamente visualizar o impacto dessas alterações. Esse ciclo contínuo de consulta e visualização estimula a exploração dos dados e a formulação de hipóteses, aproximando o trabalho técnico do processo analítico propriamente dito.

4.2 Recursos de modelagem nativos do SQL Server

Modelagem direta e geração de diagramas dentro do ambiente do SQL Server

Diferentemente de processos que se iniciam em ferramentas conceituais externas, essa abordagem parte da definição efetiva das estruturas físicas do banco de dados, permitindo que o modelo seja construído e visualizado diretamente no ambiente onde será executado. Essa proximidade entre modelagem e implementação reduz discrepâncias, favorece a consistência estrutural e aproxima o projetista da realidade operacional do sistema.

A modelagem direta baseia-se na criação de tabelas, colunas, chaves e relacionamentos como ponto de partida do processo de design. Ao definir essas estruturas diretamente no banco de dados, o profissional transforma conceitos abstratos em elementos concretos, passíveis de validação imediata.

A geração de diagramas a partir dessas estruturas físicas permite visualizar o modelo relacional já implementado, oferecendo uma representação gráfica que facilita a análise e a compreensão das relações entre as entidades.

A integração entre modelagem direta e geração de diagramas reforça a ideia de que o banco de dados é um artefato vivo, em constante evolução. Em vez de tratar o modelo como um documento estático criado apenas no início do projeto, essa abordagem permite que o modelo seja continuamente ajustado e visualizado à medida que novas tabelas são criadas, relacionamentos são definidos e regras de integridade são refinadas.

A modelagem direta não elimina a importância do planejamento conceitual, mas redefine sua aplicação no processo de desenvolvimento. Em muitos casos, o planejamento ocorre de forma iterativa e distribuída, sendo refinado à medida que o banco de dados evolui. A geração de diagramas nesse contexto atua como mecanismo de feedback visual, permitindo avaliar se a implementação está alinhada aos conceitos pretendidos.

O processo prático de modelagem direta no ambiente do SQL Server caracteriza-se pela criação incremental das estruturas de dados, nas quais tabelas, colunas e relacionamentos são definidos progressivamente à medida que os requisitos do sistema se tornam mais claros. Essa abordagem permite que o modelo evolua de forma orgânica, acompanhando as necessidades do negócio e incorporando ajustes conforme novas informações são obtidas.

Unidade III

5 OTIMIZAÇÃO DE CONSULTAS EM SQL SERVER

O uso adequado de índices e a aplicação de ajustes de performance assumem papel central na otimização do SQL Server. A criação e a utilização correta de índices permitem acelerar o acesso aos dados, enquanto o monitoramento das queries e a análise do comportamento do banco possibilitam identificar operações custosas e oportunidades de melhoria.

5.1 Índices e performance de consultas no SQL Server

Criação e utilização de índices para melhorar o desempenho

O desempenho das consultas em um banco de dados relacional está diretamente relacionado à forma como os dados são armazenados, organizados e acessados pelo sistema gerenciador. No contexto do SQL Server, os índices desempenham papel central nesse processo, pois atuam como estruturas auxiliares que permitem localizar informações de forma mais eficiente, reduzindo o volume de dados que precisa ser examinado durante a execução de uma consulta.

Índices podem ser entendidos como mecanismos de acesso que organizam referências aos dados armazenados nas tabelas, permitindo que o SQL Server encontre rapidamente os registros relevantes sem a necessidade de percorrer toda a tabela.

A utilização de índices transforma o meio como o mecanismo de execução do SQL Server que processa uma consulta. Em vez de acessar diretamente os dados brutos, o sistema consulta o índice, que contém informações organizadas de maneira otimizada para buscas. Essa organização permite reduzir drasticamente o número de operações de leitura necessárias, resultando em ganhos significativos de desempenho, especialmente em consultas que envolvem filtros, junções e ordenações frequentes.

A criação de índices no SQL Server exige compreensão não apenas de sua finalidade, mas também de sua estrutura interna e de seu impacto sobre outras

operações do banco de dados. Embora os índices aceleram consultas de leitura, eles também introduzem custos adicionais em operações de inserção, atualização e exclusão, pois precisam ser mantidos e atualizados sempre que os dados subjacentes são modificados.

A análise das consultas executadas com maior frequência é um ponto de partida fundamental para a definição de uma estratégia de indexação eficaz. Consultas críticas, que impactam diretamente a experiência do usuário ou o desempenho geral do sistema, devem ser priorizadas na criação de índices.

Os índices são construídos a partir das colunas que identificam e relacionam registros, influenciando diretamente a forma como os dados são localizados e acessados nas tabelas.

O índice clustered ocupa posição central na organização física dos dados de uma tabela. Nesse tipo de índice, os dados são armazenados fisicamente na ordem definida pela chave do índice, o que significa que a própria tabela é organizada conforme esse critério. Essa característica torna o índice clustered eficiente para consultas que envolvem intervalos de valores, ordenações frequentes ou recuperação sequencial de dados.

A definição do índice clustered deve considerar cuidadosamente a coluna ou conjunto de colunas que melhor representa o padrão de acesso predominante à tabela. Colunas utilizadas frequentemente em filtros, junções ou ordenações são candidatas naturais a essa função, desde que apresentem valores suficientemente distintos para evitar concentração excessiva de registros.

Os índices nonclustered, por sua vez, constituem estruturas independentes da organização física da tabela. Eles armazenam uma cópia ordenada das chaves indexadas, juntamente com referências que apontam para a localização dos dados reais. Essa independência permite que uma tabela possua múltiplos índices nonclustered, cada um atendendo a diferentes padrões de consulta. Essa flexibilidade torna os índices nonclustered amplamente utilizados para acelerar consultas específicas sem alterar a ordem física dos dados. A utilização de índices nonclustered é vantajosa em consultas que filtram ou agrupam dados com base em colunas não utilizadas como chave primária ou clustered. Ao criar índices direcionados a essas colunas, o SQL Server consegue localizar rapidamente os registros relevantes, reduzindo o custo de leitura e melhorando o tempo de resposta.

No entanto, o excesso de índices nonclustered pode aumentar o custo de manutenção e impactar negativamente as operações de escrita.

Os índices compostos, formados por mais de uma coluna, representam uma estratégia para atender consultas que utilizam múltiplos critérios de filtragem ou ordenação. A ordem das colunas em um índice composto influencia diretamente sua eficácia, pois determina como o índice será utilizado pelo otimizador de consultas. Colunas mais seletivas ou mais frequentemente utilizadas nos filtros iniciais devem ser posicionadas de forma estratégica para maximizar os benefícios do índice.

A definição de índices deve levar em conta a cardinalidade das colunas, isto é, a quantidade de valores distintos presentes. Colunas com alta cardinalidade tendem a se beneficiar mais da indexação, pois permitem filtrar um conjunto menor de registros com maior precisão. Já colunas com baixa cardinalidade podem oferecer ganhos limitados, pois muitos registros compartilham os mesmos valores, reduzindo a seletividade do índice.

A análise de planos de execução constitui um dos instrumentos mais importantes para compreender como o SQL Server utiliza os índices disponíveis e como as consultas são processadas internamente. O plano de execução descreve, de forma detalhada, as etapas que o mecanismo de banco de dados seguirá para recuperar os dados solicitados, incluindo a escolha dos índices, o tipo de acesso aos dados e a ordem das operações. Ao interpretar corretamente essas informações, o profissional consegue identificar gargalos de desempenho e ajustar a estratégia de indexação de maneira fundamentada.

O plano de execução revela se o SQL Server está realizando varreduras completas de tabelas ou se está utilizando índices de forma eficiente. Quando uma consulta utiliza um índice adequado, o plano tende a mostrar operações de busca direcionada, que acessam apenas os registros necessários. Por outro lado, a presença frequente de varreduras completas pode indicar ausência de índices apropriados ou consultas formuladas de maneira que inviabiliza seu uso.

A interpretação do plano de execução exige compreensão dos operadores utilizados pelo mecanismo de consulta. Cada operador representa uma etapa específica do processamento, como leitura de dados, junções ou ordenações. Ao observar a sequência desses operadores, o profissional pode identificar quais etapas consomem mais recursos e quais poderiam ser otimizadas por meio da criação ou ajuste de índices.

O equilíbrio entre operações de leitura e escrita é outro aspecto central na definição de estratégias de indexação e manutenção. Embora índices acelerem consultas de leitura, eles aumentam o custo das operações de escrita, pois cada modificação nos dados exige atualização das estruturas indexadas. Em sistemas com alta taxa de escrita, a criação excessiva de índices pode resultar em degradação do desempenho geral, exigindo uma análise cuidadosa das prioridades do sistema.

5.2 Ajustes de performance no SQL Server

Monitoramento de queries e otimização de banco de dados

As ferramentas e as métricas de monitoramento disponíveis no SQL Server fornecem os meios necessários para observar, analisar e compreender o comportamento das queries em execução, constituindo a base técnica para qualquer ajuste de performance eficaz. O monitoramento adequado transforma o desempenho do banco de dados em um fenômeno observável e mensurável, permitindo que decisões de otimização sejam tomadas com base em evidências concretas e não apenas em percepções subjetivas de lentidão.

Uma das principais contribuições do monitoramento é a capacidade de identificar quais consultas estão sendo executadas com maior frequência e quais consomem mais recursos ao longo do tempo. Consultas aparentemente simples, quando executadas milhares de vezes, podem exercer impacto significativo sobre o desempenho global do sistema. O acompanhamento sistemático dessas execuções permite detectar padrões de uso e direcionar esforços de otimização para as áreas de maior impacto.

O monitoramento do consumo de CPU pelas queries é relevante em ambientes com múltiplas aplicações concorrentes. Consultas intensivas em processamento podem monopolizar recursos e degradar o desempenho de outras operações. Identificar essas consultas permite avaliar se o custo computacional é justificado ou se ajustes na lógica da query, na indexação ou na estrutura do banco de dados podem reduzir o impacto sobre o sistema.

As métricas relacionadas ao tempo de execução das queries são fundamentais para avaliar o desempenho percebido pelo usuário. O tempo total de resposta, o tempo de CPU consumido e o tempo de espera por recursos oferecem perspectivas complementares sobre como uma consulta se comporta no ambiente. A análise

conjunta dessas métricas permite distinguir entre consultas limitadas por processamento, por acesso a disco ou por contenção de recursos, orientando a escolha das técnicas de otimização mais adequadas.

O monitoramento do consumo de CPU pelas queries é relevante em ambientes com múltiplas aplicações concorrentes. Consultas intensivas em processamento podem monopolizar recursos e degradar o desempenho de outras operações. Identificar essas consultas permite avaliar se o custo computacional é justificado ou se ajustes na lógica da query, na indexação ou na estrutura do banco de dados podem reduzir o impacto sobre o sistema. Enquanto operações com muitas gravações podem estar associadas a processos de carga ou atualização mal otimizados. O monitoramento desses aspectos ajuda a compreender como as queries interagem com o subsistema de armazenamento e a identificar oportunidades de redução de acesso a disco.

O monitoramento permite avaliar a variabilidade do desempenho das queries ao longo do tempo. Consultas que apresentam comportamento inconsistente, com tempos de resposta variando significativamente entre execuções, podem indicar problemas relacionados a planos de execução instáveis ou a mudanças na distribuição dos dados. Identificar essa variabilidade é essencial para promover ajustes que resultem em desempenho mais previsível e confiável.

A coleta e a análise histórica de métricas de performance ampliam ainda mais a eficácia do monitoramento.

O monitoramento das queries auxilia na avaliação do impacto de mudanças realizadas no banco de dados ou nas aplicações. Após a implementação de novas funcionalidades, alterações estruturais ou ajustes de configuração, a observação das métricas de desempenho permite verificar se essas mudanças afetaram positiva ou negativamente o comportamento do sistema.

A correlação entre métricas de monitoramento e planos de execução aprofunda a compreensão do desempenho das queries, ao relacionar dados de consumo de recursos com a forma como o banco de dados executa as consultas se identifica com maior precisão as causas dos problemas de performance.

A otimização no nível das consultas demonstra que pequenas mudanças na forma como o SQL é escrito podem gerar ganhos expressivos de desempenho. A eliminação de operações desnecessárias, o uso adequado de filtros e junções e a redução do volume de dados processados são estratégias que, quando aplicadas

com base em análise cuidadosa, melhoram significativamente a eficiência das queries. Essas otimizações reforçam a importância da qualidade do código SQL como componente essencial da performance do banco de dados.

A otimização estrutural envolve avaliar a organização física dos dados no armazenamento. A forma como tabelas e índices são distribuídos em discos e arquivos influencia o desempenho das operações de leitura e escrita. Embora muitos detalhes dessa organização sejam gerenciados automaticamente pelo sistema, decisões relacionadas ao planejamento de armazenamento e à distribuição de arquivos podem impactar significativamente a performance em ambientes de grande escala.

A manutenção dos índices é outro componente essencial da otimização estrutural. Com o passar do tempo, operações de modificação dos dados podem gerar fragmentação, reduzindo a eficiência das estruturas de indexação.

O monitoramento fornece a visibilidade necessária para identificar problemas, enquanto a otimização transforma essa visibilidade em ações concretas de melhoria. Juntas, essas práticas constituem a base de uma estratégia consistente de ajuste de performance, monitoramento de queries, a análise de gargalos e a otimização estrutural, o profissional amplia sua capacidade de construir e manter sistemas eficientes, escaláveis e confiáveis.

6.1 Desenvolvimento de uma solução relacional completa no SQL Server

Projeto de uma solução usando SQL Server

Considere o desenvolvimento de um sistema de pedido de vendas utilizado por uma empresa comercial para registrar clientes, controlar pedidos, gerenciar produtos e acompanhar os pagamentos realizados.

No centro desse sistema está a entidade Cliente, responsável por armazenar os dados das pessoas ou empresas que realizam compras. Os pedidos efetuados pelos clientes são representados pela entidade Pedido. Como todo pedido está necessariamente associado a um cliente, a entidade Pedido contém o atributo `id_cliente`, que funciona como chave estrangeira e estabelece o vínculo entre o pedido e o cliente que o realizou.

Os produtos comercializados pela empresa são armazenados na entidade Produto, que reúne informações essenciais para a operação do negócio. Um mesmo pedido pode conter vários produtos, e um mesmo produto pode aparecer em diferentes pedidos. Para representar corretamente essa relação muitos-para-muitos entre Pedido e Produto, utiliza-se a entidade ItemPedido. Essa entidade registra cada item que compõe um pedido, indicando qual produto foi vendido, em qual pedido ele está incluído e em que quantidade.

A entidade FormaPagamento representa os meios pelos quais os clientes podem quitar seus pedidos, como cartão de crédito, boleto bancário, transferência eletrônica ou pagamento em dinheiro.

Entidade-relacionamento (DER):

- Cliente:

- id_cliente (identificador único); — nome; — e-mail; — telefone; — endereço.

- Pedido: — id_pedido (identificador único); — data_pedido; — id_cliente (referência ao cliente que realizou o pedido).

- Produto: — id_produto (identificador único); — nome_produto; — preco_unitario; — categoria.

- ItemPedido (representa os produtos que compõem um pedido): — id_item (identificador único); — id_pedido (referência ao pedido); — id_produto (referência ao produto); — quantidade; — preco_total.

- FormaPagamento: — id_forma (identificador único); — descrição.

- Pagamento: — id_pagamento (identificador único); — id_pedido (referência ao pedido); — id_forma (referência à forma de pagamento); — valor_pago; — data_pagamento.

Modelo conceitual

No modelo conceitual, são representados:

- Entidades: — Cliente; — Pedido; — Produto; — ItemPedido; — FormaPagamento; — Pagamento.

- Relacionamentos: — Cliente realiza Pedido (1:N); — Pedido possui ItemPedido (1:N); — Produto compõe ItemPedido (1:N); — Pedido está associado a Pagamento (1:1 ou 1:N, dependendo do negócio); — Pagamento é feito via FormaPagamento (N:1).

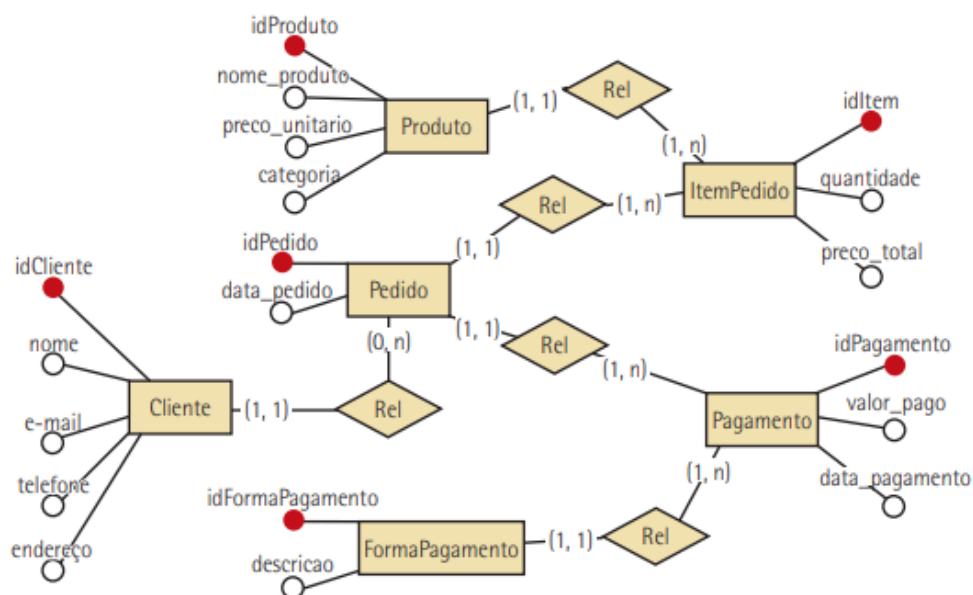


Figura 7 – Modelo conceitual

Modelo lógico

Convertemos os relacionamentos em estruturas de tabelas relacionais e definimos chaves primárias (PK) e chaves estrangeiras (FK):

- Cliente (PK: id_cliente);
- Pedido (PK: id_pedido, FK: id_cliente → Cliente.id_cliente);
- Produto (PK: id_produto);
- ItemPedido (PK: id_item, FK: id_pedido → Pedido.id_pedido, FK: id_produto → Produto.id_produto);
- FormaPagamento (PK: id_forma);
- Pagamento (PK: id_pagamento, FK: id_pedido → Pedido.id_pedido, FK: id_forma → FormaPagamento.id_forma).

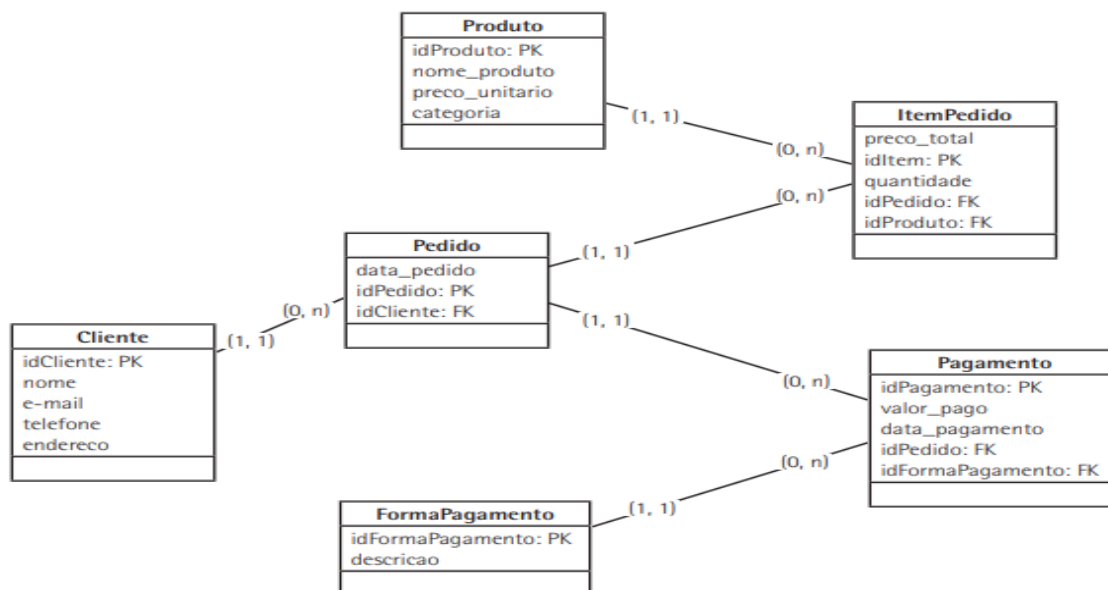


Figura 8 – Modelo lógico

Podemos criar no brModelo o script para gerar o modelo físico no SGBD.

```
CREATE TABLE Produto
(
  idProduto INT PRIMARY KEY,
  nome_produto INT,
  preco_unitario INT,
  categoria INT,
);
```

Unidade III

```
CREATE TABLE Pedido
(
  data_pedido INT,
  idPedido INT PRIMARY KEY,
  idCliente INT,
);

CREATE TABLE ItemPedido
(
  preco_total INT,
  idItem INT PRIMARY KEY,
  quantidade INT,
  idPedido INT,
  idProduto INT,
);

CREATE TABLE Cliente
(
  idCliente INT PRIMARY KEY,
  nome INT,
  e-mail INT,
  telefone INT,
  endereco INT,
);

CREATE TABLE FormaPagamento
(
  idFormaPagamento INT PRIMARY KEY,
  descricao INT,
);

CREATE TABLE Pagamento
(
  idPagamento INT PRIMARY KEY,
  valor_pago INT,
  data_pagamento INT,
  idPedido INT,
  idFormaPagamento INT,
);

ALTER TABLE Pedido ADD FOREIGN KEY(idCliente) REFERENCES Cliente (idCliente)

ALTER TABLE ItemPedido ADD FOREIGN KEY(idPedido) REFERENCES Pedido (idPedido)

ALTER TABLE ItemPedido ADD FOREIGN KEY(idProduto) REFERENCES Produto (idProduto)

ALTER TABLE Pagamento ADD FOREIGN KEY(idPedido) REFERENCES Pedido (idPedido)

ALTER TABLE Pagamento ADD FOREIGN KEY(idFormaPagamento) REFERENCES
FormaPagamento (idFormaPagamento)
```

Script do SGBD:

```
CREATE TABLE Cliente (  
    id_cliente INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    telefone VARCHAR(20),  
    endereco VARCHAR(200)  
);  
  
CREATE TABLE Pedido (  
    id_pedido INT PRIMARY KEY,  
    data_pedido DATE NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    id_cliente INT NOT NULL,  
    FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente) ON DELETE CASCADE  
);  
  
CREATE TABLE Produto (  
    id_produto INT PRIMARY KEY,  
    nome_produto VARCHAR(100) NOT NULL,  
    preco_unitario DECIMAL(10,2) NOT NULL CHECK (preco_unitario >= 0),  
    categoria VARCHAR(50)  
);  
  
CREATE TABLE ItemPedido (  
    id_item INT PRIMARY KEY,  
    id_pedido INT NOT NULL,  
    id_produto INT NOT NULL,  
    quantidade INT NOT NULL CHECK (quantidade > 0),  
    subtotal DECIMAL(10,2) NOT NULL CHECK (subtotal >= 0),  
    FOREIGN KEY (id_pedido) REFERENCES Pedido(id_pedido) ON DELETE CASCADE,  
    FOREIGN KEY (id_produto) REFERENCES Produto(id_produto)  
);  
  
CREATE TABLE FormaPagamento (  
    id_forma INT PRIMARY KEY,  
    descricao VARCHAR(50) NOT NULL UNIQUE  
);  
  
CREATE TABLE Pagamento (  
    id_pagamento INT PRIMARY KEY,  
    id_pedido INT NOT NULL,  
    id_forma INT NOT NULL,  
    valor_pago DECIMAL(10,2) NOT NULL CHECK (valor_pago >= 0),  
    data_pagamento DATE NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (id_pedido) REFERENCES Pedido(id_pedido),  
    FOREIGN KEY (id_forma) REFERENCES FormaPagamento(id_forma)  
);
```

SSMS, que é uma ferramenta gráfica gratuita, fornecida pela Microsoft, utilizada para gerenciar instâncias do Microsoft SQL Server. Ela oferece uma interface completa para administração, configuração, desenvolvimento de consultas SQL e manutenção de bancos de dados, sendo adotada por DBAs, desenvolvedores e analistas de dados.

Com o SSMS, é possível:

- Criar, alterar e excluir bancos de dados e suas estruturas (tabelas, views, procedimentos armazenados etc.).
- Executar scripts SQL e consultar dados de maneira interativa.
- Monitorar o desempenho do servidor e

analisar planos de execução. • Gerenciar logins, permissões e políticas de segurança. • Realizar backups, restaurar bancos e automatizar tarefas via Agente SQL Server

6.2 Implementação de consultas complexas no SQL Server

Desenvolvimento de consultas complexas e uso de funções avançadas

Em um sistema de pedido de vendas, o banco de dados não serve apenas para registrar clientes, pedidos e pagamentos, mas também para produzir informações que auxiliem gestores a acompanhar resultados, identificar padrões e tomar decisões estratégicas. Para atingir esse objetivo, é necessário empregar consultas SQL mais sofisticadas, capazes de consolidar dados de várias tabelas, realizar cálculos automáticos e aplicar regras de negócio diretamente no SGBD.

Uma das primeiras necessidades nesse cenário consiste em verificar a situação financeira de cada pedido. A empresa precisa saber qual foi o valor total vendido, quanto já foi pago e se ainda existe saldo pendente. Para resolver esse problema, a consulta começa criando duas expressões de tabela comuns, chamadas CTEs, por meio da cláusula WITH. A primeira delas soma os valores registrados na tabela ItemPedido, produzindo o total de cada venda, enquanto a segunda agrupa os pagamentos para obter quanto já foi quitado. Em seguida, esses resultados intermediários são combinados com os dados do cliente.

```

;WITH TotaisPedido AS (
    SELECT
        p.id_pedido,
        p.id_cliente,
        p.data_pedido,
        SUM(ip.preco_total) AS total_pedido
    FROM Pedido p
    INNER JOIN ItemPedido ip
        ON ip.id_pedido = p.id_pedido
    GROUP BY
        p.id_pedido, p.id_cliente, p.data_pedido
),

TotaisPagamento AS (
    SELECT
        pg.id_pedido,

```

MODELAGEM DE BANCO DE DADOS E NOSQL

```

        SUM(pg.valor_pago) AS total_pago
    FROM Pagamento pg
    GROUP BY pg.id_pedido
)

SELECT
    tp.id_pedido,
    tp.data_pedido,
    c.nome AS nome_cliente,
    tp.total_pedido,
    COALESCE(tpg.total_pago, 0) AS total_pago,
    tp.total_pedido - COALESCE(tpg.total_pago, 0) AS saldo,
    CASE
        WHEN COALESCE(tpg.total_pago, 0) = 0 THEN 'NAO PAGO'
        WHEN COALESCE(tpg.total_pago, 0) < tp.total_pedido THEN 'PARCIAL'
        ELSE 'PAGO'
    END AS status
FROM TotaisPedido tp
INNER JOIN Cliente c
    ON c.id_cliente = tp.id_cliente
LEFT JOIN TotaisPagamento tpg
    ON tpg.id_pedido = tp.id_pedido;

```

Outro tipo de análise bastante comum é a identificação dos clientes que mais faturam. O objetivo agora é ordenar os compradores de acordo com o total gasto, facilitando decisões comerciais e programas de fidelização.

```

;WITH FaturamentoCliente AS (
    SELECT
        p.id_cliente,
        SUM(ip.preco_total) AS faturamento
    FROM Pedido p
    INNER JOIN ItemPedido ip
        ON ip.id_pedido = p.id_pedido
    GROUP BY p.id_cliente
)

SELECT
    c.nome,
    fc.faturamento,
    DENSE_RANK() OVER (ORDER BY fc.faturamento DESC) AS posicao

```

Unidade III

```

FROM FaturamentoCliente fc
INNER JOIN Cliente c
    ON c.id_cliente = fc.id_cliente;

```

DENSE_RANK() pertence ao conjunto das window functions e trabalha sobre um conjunto de linhas definido pela cláusula OVER, permitindo criar classificações sem eliminar registros do resultado.

```

SELECT
    ip.id_pedido,
    pr.nome_produto,
    ip.preco_total,
    SUM(ip.preco_total)
        OVER (PARTITION BY ip.id_pedido) AS total_pedido,
    CAST(
        100.0 * ip.preco_total /
        NULLIF(
            SUM(ip.preco_total)
                OVER (PARTITION BY ip.id_pedido),
            0
        )
        AS DECIMAL(10,2)
    ) AS percentual
FROM ItemPedido ip
INNER JOIN Produto pr
    ON pr.id_produto = ip.id_produto;

```

O PARTITION BY reinicia o cálculo para cada pedido, NULLIF evita divisão por zero e CAST controla o formato do valor percentual.

```

;WITH FaturamentoMensal AS (
    SELECT
        DATEFROMPARTS(YEAR(p.data_pedido), MONTH(p.data_pedido), 1) AS mes_ref,
        SUM(ip.preco_total) AS faturamento_mes
    FROM Pedido p
    INNER JOIN ItemPedido ip
        ON ip.id_pedido = p.id_pedido
    GROUP BY DATEFROMPARTS(YEAR(p.data_pedido), MONTH(p.data_pedido), 1)
)

SELECT
    mes_ref,
    faturamento_mes,
    SUM(faturamento_mes) OVER (ORDER BY mes_ref) AS acumulado
FROM FaturamentoMensal
ORDER BY mes_ref;

```

SQL Server permite encapsular regras de negócio em funções definidas pelo usuário. Um exemplo típico nesse sistema é o cálculo do total de um pedido, frequentemente reutilizado em relatórios. Para isso, cria-se uma função escalar que recebe o identificador do pedido e devolve o somatório de seus itens.

```

CREATE OR ALTER FUNCTION dbo.fn_total_pedido (@id_pedido INT)
RETURNS DECIMAL(18,2)
AS
BEGIN
    DECLARE @total DECIMAL(18,2);

    SELECT @total = COALESCE(SUM(preco_total), 0)
    FROM ItemPedido
    WHERE id_pedido = @id_pedido;

    RETURN @total;
END;
GO

```

Outro recurso útil é a função tabelada, que retorna um conjunto de registros. No contexto do sistema de pedido de vendas, ela pode ser usada para listar os itens de um pedido específico, funcionando como uma view parametrizada.

```

CREATE OR ALTER FUNCTION dbo.fn_itens_pedido (@id_pedido INT)
RETURNS TABLE
AS
RETURN
(
    SELECT
        ip.id_item,
        pr.nome_produto,
        ip.quantidade,
        ip.preco_total
    FROM ItemPedido ip
    INNER JOIN Produto pr
        ON pr.id_produto = ip.id_produto
    WHERE ip.id_pedido = @id_pedido
);
GO

```


SQL Server pode ser explorado para transformar dados operacionais do sistema de pedido de vendas em informações consolidadas, rankings, percentuais e séries históricas.

Unidade IV